# Advanced Big Data Analytics Assignment I

*Smart Energy Grid Analytics*

**Issued 1 October 2026  Due 15 October 2026 at 11:59 PM CAT**

**Up to 3 students per group  Total 100 marks**

You are data analysts at an electricity utility. Use readings from 4,000 smart meters to investigate consumption, forecast demand, and recommend storage and dashboard configurations.

This project requires up to 10 GB of available disk space. All dates and times use CAT. Date ranges include both named days unless an exclusive endpoint is stated.

Embed each required screenshot beside its subsection. Show the executed command or action, its result, the taskbar, and the clock. The only supplied code is smart_meter_simulator.py. Write your own processing and database code.

## 1 Build the ingestion pipeline

### 1.1 Confirm the environment [3 marks]

1. Start your installed EMQX and PostgreSQL services. Use TimescaleDB 2.10 or later with a compatible PostgreSQL version. Create one PostgreSQL database for this assignment and enable TimescaleDB in it. Use this database for all tables and views below. Set its session time zone to Africa/Kigali and align daily buckets to CAT midnight.
2. Record CPU model and core count, RAM, OS and version, storage type, and Python, EMQX, PostgreSQL, and TimescaleDB versions. Record shared_buffers, work_mem, max_parallel_workers_per_gather, and jit. State whether PostgreSQL runs natively, in a container, or in a virtual machine, including any CPU or RAM limits.
3. Keep the machine clock synchronized and record actual execution times in CAT. Include screenshots of the running broker, connection to the assignment database, enabled extension, and session time zone.

### 1.2 Create the database structures [3 marks]

1. In PostgreSQL, create energy_live as a regular table and energy_readings as a TimescaleDB hypertable partitioned by event_time with one-day chunks.
2. Give both reading tables eight columns. Use BIGINT for meter_id, TIMESTAMPTZ for event_time and received_time, and DOUBLE PRECISION for power_kw, voltage_v, current_a, frequency_hz, and energy_kwh. Allow duplicate readings at this stage.
3. Create a third PostgreSQL table named meters. Give it meter_id as a BIGINT primary key, region and customer_type as TEXT, and base_power_kw and power_factor as DOUBLE PRECISION.
4. Write a Python program that imports smart_meter_simulator, calls meter_metadata(), and inserts each returned dictionary as one row in meters. Verify that meters contains 4,000 rows.
5. In the report, provide a data dictionary listing each table and column, its PostgreSQL type, meaning, and unit. Include screenshots of all three tables showing column names and types, the hypertable chunk interval, and the query confirming 4,000 metadata rows.

Each reading covers 15 minutes. event_time is the interval start. received_time is simulated arrival, not database insertion time. power_kw is interval average power. energy_kwh is interval energy, approximately power_kw multiplied by 0.25 hours. power_factor has no unit.

### 1.3 Capture and publish a pilot stream [4 marks]

1. Write a Python publisher that imports the unchanged generator, takes the first 2,000 dictionaries from iter_readings(), converts each to JSON, and publishes one message to energy/meters/{meter_id}. Substitute the reading's meter ID in the topic. Use MQTT QoS 1 without retained messages.
2. Write a Python subscriber for energy/meters/# at QoS 1. It must parse each JSON message and insert its eight fields into energy_live. Begin with an empty energy_live table and start the subscriber before the publisher.
3. Count dictionaries taken from the generator, messages acknowledged by the broker, and messages received by the subscriber. After all database inserts are committed, query the number of rows in energy_live. Present these four counts in one report table. Count deliberate generator duplicates as separate messages and rows.
4. The target is 2,000 at each stage. For any difference, state its size and use logs to identify losses, redelivery, parsing errors, or failed inserts. Correct pipeline faults and repeat the pilot.
5. Include screenshots of publisher completion and acknowledged count, subscriber received count, and database queries showing the stored count and sample rows. The pilot replays historical readings rather than waiting 15 minutes between records.

## 2 Capture and prepare the historical dataset

### 2.1 Capture and load the complete stream [4 marks]

1. Call dataset_info() from your Python program and record its expected counts. Start a fresh iter_readings() iterator. It covers 4,000 meters every 15 minutes from 00:00 on 1 September to 00:00 on 29 September 2026, exclusive, with 10,752,000 planned readings.
2. Write your own program to capture the full stream as CSV for bulk loading, or pass it through your own MQTT or direct database writer. Process records progressively in batches. Load all emitted rows into the initially empty energy_readings table, including duplicates. Leave the pilot in energy_live.
3. Display progress and the final number of dictionaries consumed. Compare that number and the committed energy_readings row count with expected_counts.emitted_readings from dataset_info(). All three must agree before cleaning. If using CSV, also count its data rows. If using MQTT, also report acknowledged and received counts as in 1.3.
4. Report the chosen ingestion route, total elapsed seconds from starting generation to committing the final row, and load seconds from the first database write to the final commit. For a CSV route, report file generation and database loading separately. These measurements concern your chosen route only.
5. Query the distinct meter count, earliest and latest event_time, and total size of energy_readings including all its chunks. Report table data and index sizes separately and their combined size in MB or GB. Include screenshots of the completed generation, capture or transfer, load, and database checks.
6. After verifying the complete database load and retaining the required screenshots, you may delete intermediate CSV files. Keep the programs needed to regenerate them.

### 2.2 Investigate data quality [3 marks]

1. Query energy_readings before cleaning. A unique reading is one meter_id and event_time pair. Count missing pairs from the planned grid, extra copies of observed pairs, and delayed unique readings. Normal arrival is the interval end, 15 minutes after event_time. Delayed arrivals occur 30 to 180 minutes after event_time.
2. Calculate missing percentage = 100 * missing pairs / planned readings. Calculate duplicate percentage = 100 * extra copies / observed unique pairs. Calculate delayed percentage = 100 * delayed unique readings / observed unique pairs. Completeness = 100 * observed unique pairs / planned readings.
3. Compare measured counts with the reference table below. Check that unique readings plus extra copies equals emitted readings, and planned readings minus unique readings equals missing readings. List the five meters with the most missing readings.
4. Include screenshots of the diagnostic queries and their results.

**Count reference. planned_readings is top-level in dataset_info(). The other five entries are inside expected_counts.**

| Entry | Expected count | Meaning |
|---|---|---|
| planned_readings | 10,752,000 | 4,000 meters times 28 days times 96 intervals. |
| emitted_readings | 10,696,848 | All yielded dictionaries, including duplicate copies. |
| unique_readings | 10,643,735 | Distinct observed meter_id and event_time pairs. |
| duplicate_readings | 53,113 | Extra copies beyond one row per observed pair. |
| delayed_unique_readings | 213,090 | Unique pairs with arrival after the interval end. |
| missing_readings | 108,265 | Planned pairs with no generated reading. |

### 2.3 Clean and preserve a consistent benchmark [3 marks]

1. In energy_readings, find rows sharing meter_id and event_time. Compare their received_time, power_kw, voltage_v, current_a, frequency_hz, and energy_kwh. If all six values match, retain one row and delete the additional copies using repeatable code. Investigate conflicting values before deletion. Keep missing intervals as gaps.
2. Report row count and summed energy_kwh before and after cleaning. Verify that the final row count matches expected_counts.unique_readings and that no duplicate pairs remain. Explain why missing readings must not automatically become zero consumption. Use this cleaned dataset for every later storage experiment.
3. Include screenshots of the cleaning completion and before-and-after results. Retain your capture and cleaning code so the common dataset can be reproduced.

## 3 Compare chunk intervals

### 3.1 Create and verify three configurations [3 marks]

1. Keep energy_readings as the one-day hypertable. In the same database, create two additional TimescaleDB hypertables named energy_readings_3h and energy_readings_week. Use the same eight columns and event_time partitioning, with three-hour and one-week chunks respectively.
2. Copy the cleaned rows into both new hypertables. On all three, use matching non-unique indexes on event_time and on meter_id followed by event_time. Keep index methods and sort orders identical.
3. Query each table's row count, distinct meter count, earliest and latest event_time, sum of power_kw, and sum of energy_kwh. Require 10,643,735 rows and 4,000 meters in each. Dates must match exactly. Allow at most 0.001 difference in each numeric sum for floating-point rounding.
4. Include screenshots showing each chunk interval, chunk count, date ranges, indexes, and matching data checks.

### 3.2 Implement and time the five workloads [4 marks]

1. Write the five SQL queries below and run each on all three hypertables. Change only the source table. Verify matching results, allowing 0.001 numeric rounding differences. Update table statistics with ANALYZE after loading so the planner has current row estimates.
2. Use one machine and one named client for all SQL configurations. Keep settings unchanged, close competing workloads, and pause loaders and assignment background jobs. Run SQL benchmarks sequentially. During measurements in Sections 3, 4, and 7, allow only the database workload being measured.
3. For each query and table, run one warm-up followed by five measured executions. Measure milliseconds from statement submission until the last result row is received, using the same elapsed-time timer. Record each run's CAT start and end, duration, and machine identifier. Report median, minimum, and maximum per query and table. Also report session seconds from the first warm-up submission to the last measured result.
4. Label results as warm-cache measurements because a warm-up precedes timing. Do not restart PostgreSQL or clear caches between measured runs. Record the client and timing tool names in the report.
5. Include screenshots of the completed timing comparison and one execution plan with actual timing and buffer statistics per configuration.

**Query workloads for 3.2**

| Query | Required result |
|---|---|
| Q1 | For each CAT hour on 21 September, average power_kw across all available meter readings in that hour. |
| Q2 | Return exactly ten 15-minute intervals from 22 to 28 September. Sum power_kw across meters per interval. Sort by total descending, then event_time ascending, and take the first ten rows, including when a tie crosses tenth place. |
| Q3 | Total energy per meter across the full historical period. |
| Q4 | Energy total, average power, and peak power for meter 1000000000 from 22 to 28 September. |
| Q5 | Full scan reporting row count, total energy, and average, minimum, and maximum power. |

### 3.3 Explain the chunk results [3 marks]

1. For each workload, give a short answer naming the fastest chunk interval, a numerical comparison of medians, and an explanation supported by chunk counts, time filters, or an actual query plan.
2. Refer to the evidence from 3.1 and 3.2. Distinguish observations from your explanation of their cause.

## 4 Compare compression strategies

### 4.1 Create the two compressed configurations [4 marks]

1. Choose one chunk interval from Section 3. Create two empty hypertables named energy_time_compressed and energy_meter_compressed with that interval, the same eight columns, and matching indexes.
2. For energy_time_compressed, order by event_time descending and use no segmentation. For energy_meter_compressed, use the same ordering and segment by meter_id. Use your installed TimescaleDB version's compression or columnstore settings.
3. Add a policy to each table to compress chunks whose end is more than 48 hours old. Pause both policies until the measurements in 4.3 are complete.
4. Before loading, record one cutoff timestamp equal to the experiment start time minus 48 hours. Use this same cutoff for both tables throughout the experiment.
5. Copy the cleaned data into each new table one chunk at a time. After a chunk is fully loaded, manually compress it if its end precedes the cutoff. Time each compression operation separately from loading.
6. Keep all five tables. Verify that the same chunk time ranges are compressed in both new tables and repeat the data checks from 3.1.
7. Include screenshots of both configurations, policies, compressed-chunk lists, and matching data checks.

### 4.2 Measure storage and compression cost [4 marks]

1. Use the uncompressed table with the chosen chunk interval as the baseline. Report its data, index, and total sizes and the corresponding final sizes of both compressed tables. Include all chunks, compressed and uncompressed, and their indexes.
2. For each strategy, divide the baseline total size by the compressed table's total size for the compression ratio. Percentage saved is 100 times (baseline size minus compressed size) divided by baseline size. Sum the measured chunk compression times, excluding data loading, for compression cost.
3. Alongside each ratio, report the compressed chunk count out of total chunks and the percentage of rows held in compressed chunks. Calculate the row percentage using the original row counts of those chunks divided by the full cleaned row count, multiplied by 100.
4. Include screenshots of the completed compression timings and storage statistics for both strategies.

### 4.3 Repeat the query comparison [4 marks]

1. Run Q1 to Q5 on both compressed tables using the correctness checks and timing procedure in 3.2. Present all five table configurations in one comparison.
2. For each compressed table and query, calculate percentage improvement as 100 times (baseline median minus compressed median) divided by baseline median. Use the uncompressed table with the same chunk interval as the baseline. Positive values mean faster execution.
3. Include screenshots of the completed results and actual execution plans for both compressed tables. Resume the compression policies after measurement and include a screenshot of their enabled status.

### 4.4 Select a storage configuration [3 marks]

1. Compare the effect of meter segmentation on the single-meter workload and workloads across all meters.
2. Give a short recommendation with three parts. Name the chosen chunk interval and compression strategy, cite measured storage and query results, and explain one cost or tradeoff using the compression time or workload differences.

## 5 Investigate the utility data

Use cleaned energy_readings and meters for Sections 5 to 8, except the separate test in 6.2. Group readings by event_time unless a step explicitly uses received_time.

### 5.1 Identify demand patterns [3 marks]

1. Plot total network energy per CAT day and totals for the four seven-day periods beginning 1, 8, 15, and 22 September. Sum energy_kwh for these consumption totals.
2. Find the three highest 15-minute network demand intervals across all 28 days by summing power_kw across meters at each event_time. Report their times and kW values. Explain the patterns, operational implications, and difference between kW and kWh.
3. Include screenshots of the executed analysis, charts, and peak-interval results.

### 5.2 Compare regions and customer types [3 marks]

1. Join readings to meters using meter_id. Over all 28 days, compare energy totals separately by region and by customer type. Divide each total by that group's registered meter count to calculate energy per meter. Report both totals, per-meter values, and meter counts.
2. State which differences remain after normalization and what they imply. Include screenshots of the group counts and comparison results.

### 5.3 Assess reporting quality [3 marks]

1. Calculate completeness for the whole network and each region over all 28 days. The planned count is registered meters times 28 days times 96 readings per day. Completeness is 100 times the observed unique meter and event_time pair count divided by that planned count.
2. For readings with event_time on 21 September, sum energy_kwh once using only received_time at or before 00:00 on 22 September and again using all arrivals. Report both totals and their difference in kWh and as a percentage of the final total. Explain the effects of missing and delayed readings.
3. Include screenshots of the completeness calculation and the two daily totals.

### 5.4 Investigate abnormal consumption [3 marks]

1. State your anomaly detection rule, thresholds, and reasons for choosing them. Apply the rule to cleaned readings before comparing it with the supplied event labels. Each flag must identify a meter_id and event_time.
2. Call anomaly_events() in your Python analysis program. An injected event is detected if at least one flag has its meter_id and falls at or after start_time but before end_time. Report detected and missed events out of the 40 supplied events. A false flag is a flagged reading outside every injected interval for that meter. Use labels only for evaluation.
3. Report false flags as a count and as a percentage of observed readings outside injected intervals. Also report the number of meters with at least one false flag, divided by 4,000 and multiplied by 100. Label this second measure as the percentage of meters affected by false flags. Explain missed events and false alarms.
4. Include screenshots of the executed detection, flagged readings, and comparison with injected events.

### 5.5 Write the management recommendation [3 marks]

1. After completing the remaining sections, write a recommendation of no more than 500 words.
2. Recommend operational actions, a storage and reporting configuration, and whether to use your forecast model. Support the recommendations with at least one measured benchmark result and one forecast metric, including units and the relevant baseline.

## 6 Create and investigate continuous aggregates

### 6.1 Create the three aggregate views [4 marks]

1. In PostgreSQL, create three initially unpopulated TimescaleDB continuous aggregate views over energy_readings. Group the first by 15-minute bucket and region, the second by CAT hour and region, and the third by CAT day and meter_id. For the regional views, use an inner join matching meter_id between energy_readings and meters. Keep metadata unchanged.
2. Each view must store its bucket and grouping key, sum of energy_kwh, sum of power_kw, and reading count. Set timescaledb.materialized_only to TRUE on all three views to disable real-time aggregation. Section 7 must measure queries against stored aggregates.
3. For each initially empty view, manually refresh the entire generated period. Report the refresh duration in seconds and the view's materialized data and index size. Complete these measurements before enabling refresh policies. Then add policies with a three-day lookback, a one-hour end offset, and a 15-minute schedule. These offsets are relative to the database clock, so policies alone will not populate the older September data.
4. Include screenshots of all view definitions, sample populated rows, storage and initial refresh measurements, policy settings, and background job status.

### 6.2 Test freshness and delayed arrivals [3 marks]

1. In the same assignment database, create a separate test hypertable partitioned by event_time, with event_time as TIMESTAMPTZ and energy_kwh as DOUBLE PRECISION. Create an initially unpopulated hourly continuous aggregate of energy sum and row count. Disable real-time aggregation and leave automatic refresh off during this test.
2. Record the actual CAT test time, H as the start of the most recent completed CAT hour, and H minus 48 hours. Record the event_time chosen for each insert. Insert a 1 kWh reading into the older hour. Manually refresh only that hour and record its aggregate result.
3. Insert another reading of 0.5 kWh into that older hour and a 2 kWh reading into hour H. Before refreshing, query both hours directly from the test table and from the aggregate. Repeat the aggregate query after enabling real-time aggregation.
4. Manually refresh both complete hours and query again. Confirm totals of 1.5 kWh and two rows in the older hour, and 2 kWh and one row in hour H. Explain which changes became visible at each stage and which hours the policy in 6.1 could refresh at the recorded test time.
5. Include screenshots of the table and view definitions, inserts, real-time settings, initial refresh, and raw and aggregate results at each comparison stage.

Worked timeline for 6.2. If the test starts at 10:30 CAT on 5 October, H is 09:00 on 5 October and the older hour begins at 09:00 on 3 October. Example event times are 3 October 09:10 for 1 kWh, 3 October 09:25 for 0.5 kWh, and 5 October 09:10 for 2 kWh. Use dates relative to your actual test time.

| Stage | Older hour aggregate | Hour H aggregate |
|---|---|---|
| Initial older-hour refresh, real-time off | 1 kWh, 1 row | No bucket returned |
| After both new inserts, before refresh, real-time off | 1 kWh, 1 row | No bucket returned |
| Real-time on, still before refresh | 1 kWh, 1 row | 2 kWh, 1 row |
| After manual refresh of both hours | 1.5 kWh, 2 rows | 2 kWh, 1 row |

After all inserts, the raw table must show 1.5 kWh and two rows in the older hour, and 2 kWh and one row in H. An absent aggregate bucket is not a zero reading. Explain the differences between stages.

## 7 Build and compare dashboards

### 7.1 Build the Raw dashboard [3 marks]

1. Use Grafana, Dash, or an equivalent tool. Build a dashboard named Raw whose measurement queries read energy_readings directly and join meters where needed. Use filters for whole CAT days and the selected meter.
2. Provide five panels showing hourly regional energy, daily network energy, network energy in the seven-day periods defined in 5.1, daily energy for a selected meter, and completeness by region as defined in 5.3. Limit every panel to the selected date range. Label energy in kWh and completeness in percent.
3. Include a screenshot of the running dashboard with all required panels. Retain its source, configuration, and panel queries.

### 7.2 Build the Aggregate dashboard [3 marks]

1. Build a second dashboard named Aggregate with the same five panels, layout, filters, and date range. Obtain hourly regional energy from the hourly regional view, daily and weekly network energy from the 15-minute regional view, meter history from the daily meter view, and observed completeness counts from the 15-minute regional view.
2. Sum stored energy sums and reading counts when combining buckets. Calculate planned readings from registered meter counts and the selected period. Measurement queries in this mode must use the aggregate views. They may query meters for labels and registered counts.
3. Include screenshots of the running Aggregate dashboard and its panel query definitions. A separate optional pilot-message panel may query energy_live in both modes and is excluded from the comparison.

**7.3 Verify matching results [3 marks]**

1. Set both dashboards to 00:00 on 22 September through 00:00 on 29 September, exclusive. Include all regions and select meter 1000000000 for the meter-history panel.
2. Export or query the numerical data behind every panel. Match rows by bucket, region, or meter as applicable. Report each panel's compared row count and maximum absolute difference. Require identical keys and reading counts, energy differences no greater than 0.001 kWh, and completeness differences no greater than 0.000001 percentage points before rounding.
3. Correct unexplained differences before timing. Include screenshots of the paired dashboard outputs and the numerical checks for all five panels.

**7.4 Measure performance and judge the tradeoff [4 marks]**

1. Keep the filters and dates from 7.3 and pause writes and assignment background jobs during timing. Disable dashboard query-result caching. Perform one warm-up refresh followed by ten measured refreshes per mode, using the same browser, machine, and connection.
2. For every refresh, record each panel's query duration and the elapsed time from requesting refresh until all five panels finish rendering. State the measurement tool used. Report median, minimum, and maximum in milliseconds for each panel and the whole dashboard. Report aggregate storage and initial refresh cost from 6.1 alongside these timings.
3. Explain whether the speed change justifies the extra storage and refresh work. Include screenshots of the completed timing results and comparison. Resume the paused jobs after measurement.

# 8 Forecast the next day

Forecast hourly regional consumption in kWh. This is an offline forecast of observed consumption. Treat completed historical hourly totals as finalized and acknowledge the remaining missing readings when discussing limitations.

**8.1 Prepare chronological data and features [3 marks]**

1. Create a Python analysis dataset with one row per region and CAT hour, summing energy_kwh from cleaned readings. Preserve observed totals as evaluation targets. Report reading counts and your handling of any absent hours or missing predictor values.
2. Split by target timestamp. Use 1 to 14 September for training, 15 to 21 September for validation, and 22 to 28 September for testing. Report row counts and date limits for each split. Fit preprocessing only on training data.
3. For each target hour, use 00:00 at the start of its day as the forecast origin. Build calendar features and lag features available before that origin. For example, the previous day's corresponding hour is available for every target hour. A preceding hour within the forecast day is not. Exclude training rows lacking the chosen initial lookback.
4. Report each feature's name, lag or window length in hours, and earliest usable target timestamp. Report the training row count before lookback exclusions, the number excluded, and the retained count. Keep the same usable training rows across model configurations.
5. Include screenshots of dataset preparation, split sizes and dates, feature examples, and checks that every lag source timestamp precedes its forecast origin.

**8.2 Evaluate the previous-day baseline [4 marks]**

1. At 00:00 CAT on each test day, forecast its 24 hourly values using the previous day's corresponding hourly consumption. Repeat for all ten regions.
2. Produce 1,680 test predictions and calculate MAE and RMSE. Include screenshots of the completed baseline run, prediction count, sample predictions, and metrics.

### 8.3 Train and evaluate one regression model [4 marks]

1. Choose one regression model and train at least two configurations on the training split. Present one validation table with configuration ID, model settings, training-row count, validation-row count, validation MAE, and validation RMSE. State the shared predictors. Select the configuration with the lowest validation MAE and freeze its fitted model and preprocessing before testing.
2. Generate the same 1,680 test predictions as the baseline, using only information available before each daily forecast origin. Earlier completed test days may supply later lag values but must not be used to refit the model or preprocessing.
3. Include screenshots showing training completion, validation MAE for each configuration, selected settings, prediction count, and sample test predictions.

### 8.4 Judge forecast usefulness [4 marks]

1. Evaluate a horizon of 24 hourly predictions per region per test day. Compare baseline and model MAE and RMSE over all 1,680 test observations and separately for each region. For prediction error use predicted minus actual kWh. MAE averages its absolute values. RMSE is the square root of its mean squared values. Both metrics are in kWh.
2. Plot actual, baseline, and model consumption for one region over all seven test days. For each region, find its highest actual test-hour consumption, breaking ties by earliest time. Report that hour, actual consumption, and each method's prediction and absolute error.
3. State whether the model improves on the baseline and whether its errors are useful for utility planning. Discuss peak errors, missing readings, and the effect of delayed arrivals on a live forecast.
4. Include screenshots of the executed metric comparison, peak-hour error results, and forecast plot.

## 9 Submit the report and code

### 9.1 Organize the report [2 marks]

1. Submit one PDF report per group in subsection order. Include measurements, charts, observations, judgments, and the management recommendation.
2. Embed every required screenshot beside its subsection, with readable results, taskbar, and clock. Reuse earlier evidence where a later subsection asks only for interpretation.

### 9.2 Make the work reproducible [2 marks]

1. Submit a GitHub repository URL and a ZIP containing the same final files. Check that both contain the unchanged generator, your capture and ingestion programs, and your SQL files.
2. Include analysis and model code, recorded benchmark timings, the validation comparison, dashboard queries and configuration, and recorded environment details.
3. Include a README giving the execution order and commands. Keep the full generated dataset local and document how to regenerate and load it. Use placeholders for passwords and connection secrets.
4. Run the README's pilot instructions and verify the four counts from 1.3. Include a screenshot of that successful run.

### 9.3 Document contributions [1 mark]

1. List all members' names, student IDs, and individual contributions.
2. Be prepared to explain and present the group's work if called upon.

Sound findings of no performance or forecasting improvement can earn full marks when supported by correct execution, observations, and reasoning.

## Appendix A Common generator reference

Import the supplied module from your own program. It generates identical data for all groups, in memory only. Submit the file unchanged, matching the issued copy byte for byte.

| Function | Returned Python value |
|---|---|
| dataset_info() | Settings dictionary including planned_readings and nested expected_counts. See the count reference in 2.2. |
| meter_metadata() | List of 4,000 dictionaries with the five meters columns defined in 1.2. |
| iter_readings() | Iterator of eight-field reading dictionaries as defined in 1.2. Each call restarts the dataset. Consume progressively. |
| anomaly_events() | List of 40 dictionaries with meter_id, start_time, end_time, and event_type. |

Meter IDs run from 1000000000 to 1000003999. Region 1 through Region 10 each have 400 meters. Customer types are residential, commercial, and industrial. power_factor is 0.95. Timestamps use ISO text with +02:00. Event starts are inclusive and ends exclusive.

Readings arrive from the iterator in event_time order, not received_time order. Duplicate copies match in all eight fields. Interval meanings and units are defined in 1.2. No seed or settings changes are required.